

**TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**



ĐỒ ÁN XỬ LÝ NGÔN NGỮ TỰ NHIÊN

ĐỀ TÀI:

**DỊCH MÁY ANH-PHÁP / ANH-ĐỨC
VỚI
MÔ HÌNH ENCODER-DECODER LSTM**

Giảng viên hướng dẫn:

PGS.TS. NGUYỄN TUẤN ĐĂNG

Sinh viên thực hiện:

NGUYỄN TOÀN THẮNG – 3123410343

HỒ MINH TIẾN – 3123410373

Thành phố Hồ Chí Minh, tháng 12 năm 2025

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sự tri ân sâu sắc đối với các thầy cô của Trường Đại học Sài Gòn, đặc biệt là các thầy cô thuộc Khoa Công nghệ Thông tin đã tạo điều kiện cho chúng em học tập và hoàn thành đồ án lần này.

Chúng em cũng xin chân thành cảm ơn thầy Nguyễn Tuấn Đăng, giảng viên phụ trách môn học, đã tận tình hướng dẫn và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài.

Trong quá trình nghiên cứu và viết báo cáo, do kiến thức và kinh nghiệm thực tế còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý của thầy cô để hoàn thiện hơn trong các bài báo cáo và dự án tương lai.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN

DANH MỤC CÁC HÌNH

CHƯƠNG 1: GIỚI THIỆU	1
1. Bài Toán Dịch Máy	1
2. Mục Tiêu Đồ Án	1
CHƯƠNG 2: DATASET VÀ TIỀN XỬ LÝ DỮ LIỆU	2
1. Dataset Multi30K	2
2. Tokenization	2
3. Xây Dựng Vocabulary	3
4. Padding và Packing Chuỗi	3
4.1. Padding	3
4.2. Packing Và Sắp Xếp Theo Độ Dài	4
CHƯƠNG 3: KIẾN TRÚC MÔ HÌNH ENCODER-DECODER LSTM	4
1. Tổng Quan Mô Hình Encoder–Decoder LSTM	4
2. Encoder	5
2.1. Input Của Encoder	5
2.2. Embedding Layer	5
2.3. LSTM Encoder	5
3. Decoder	6
3.1. Input Ban Đầu Của Decoder	6
3.2. Teacher Forcing	6
3.3. Dự Đoán Token Tiếp Theo	6
4. Các Tham Số Mô Hình	7

5. Cơ Chế Attention (Luong Attention)	7
CHƯƠNG 4: HUẤN LUYỆN MÔ HÌNH	8
1. Hàm Mất Mát (Loss Function)	8
2. Optimizer Và Hyperparameters	9
3. Teacher Forcing Trong Huấn Luyện	9
4. Early Stopping Và Checkpoint	9
CHƯƠNG 5: DỰ ĐOÁN VÀ HÀM TRANSLATE	10
1. Greedy Decoding	10
2. Hàm Translate(Sentence)	11
CHƯƠNG 6: ĐÁNH GIÁ MÔ HÌNH	11
1. BLEU Score	11
2. Nhận Xét Kết Quả	12
CHƯƠNG 7: PHÂN TÍCH LỖI	13
1. Ví Dụ Dịch	13
2. Phân Tích Lỗi	13
CHƯƠNG 8: ĐỀ XUẤT CẢI TIẾN	14
1. Vai Trò Của Cơ Chế Attention	14
2. So Sánh Mô Hình Có Và Không Có Attention (Phân Tích Định Tính)	15
3. Sử Dụng Beam Search Thay Cho Greedy Decoding	15
4. Sử Dụng Subword Tokenization (BPE)	15
CHƯƠNG 9: KẾT LUẬN	16

CHƯƠNG 10: PHỤ LỤC	16
1. Kho Mã Nguồn Của Nhóm	16
2. Lý Do Sử Dụng Google Colab	17
3. Môi Trường Thực Nghiệm	17
4. Cách Tổ Chức Và Sử Dụng Mã Nguồn	18
5. Checkpoint Mô Hình	18
6. Một Số Đoạn Mã Nguồn Quan Trọng	18

DANH MỤC CÁC HÌNH

Danh sách hình vẽ

1	Sơ Đồ Kiến Trúc Mô Hình	4
2	Biểu Đồ Training Loss Và Validation Loss Theo Epoch	8
3	Kết quả Từng Output Epoch Của Đồ Án	10
4	Kết quả BLUE Score Từ Đồ Án	12
5	Kết Quả Từ Mô Hình Đã Huấn Luyện	13
6	So Sánh Kết Quả Giữa Mô Hình Với Dịch Chuẩn	13

CHƯƠNG 1: GIỚI THIỆU

1. Bài Toán Dịch Máy

Dịch máy (Machine Translation) là bài toán trong lĩnh vực Xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) nhằm tự động chuyển đổi một câu hoặc văn bản từ ngôn ngữ nguồn sang ngôn ngữ đích mà vẫn bảo toàn ý nghĩa. Đây là một trong những bài toán quan trọng và có tính ứng dụng cao của trí tuệ nhân tạo, được sử dụng rộng rãi trong các hệ thống như Google Translate, trợ lý ảo và các ứng dụng đa ngôn ngữ.

Trong đề tài này, nhóm tập trung vào bài toán **dịch máy từ tiếng Anh sang tiếng Pháp (English → French)**. Cặp ngôn ngữ này được lựa chọn vì các lý do sau:

- Tiếng Anh và tiếng Pháp có **khác biệt rõ rệt về ngữ pháp, trật tự từ và chia thì**, phù hợp để đánh giá khả năng học ngữ cảnh của mô hình.
- Dataset Anh – Pháp được công bố rộng rãi, đặc biệt là **Multi30K**, thuận lợi cho việc huấn luyện và đánh giá mô hình.
- Đây là cặp ngôn ngữ thường được sử dụng trong các nghiên cứu dịch máy, giúp kết quả dễ so sánh và phân tích.

Bài toán dịch máy được tiếp cận theo hướng **Sequence-to-Sequence (Seq2Seq)**^[1], trong đó một chuỗi đầu vào (câu tiếng Anh) được ánh xạ sang một chuỗi đầu ra có độ dài thay đổi (câu tiếng Pháp).

2. Mục Tiêu Đồ Án

Mục tiêu chính của đồ án là **xây dựng và triển khai hoàn chỉnh một mô hình dịch máy Encoder–Decoder sử dụng LSTM từ đầu**, không sử dụng mô hình dựng sẵn, nhằm hiểu rõ cơ chế hoạt động của từng thành phần trong hệ thống dịch máy.

Cụ thể, đồ án hướng đến các mục tiêu sau:

- **Xây dựng mô hình Encoder–Decoder dựa trên LSTM** cho bài toán dịch Anh – Pháp, bao gồm các bước tiền xử lý dữ liệu, xây dựng vocabulary, thiết kế kiến trúc mô hình và huấn luyện.

- **Đánh giá chất lượng mô hình** thông qua chỉ số **BLEU score** trên tập dữ liệu kiểm thử.
- **Phân tích lỗi dịch** thông qua các ví dụ cụ thể, từ đó chỉ ra các hạn chế của mô hình như từ ngoài từ điển (OOV), câu dài hoặc sai cấu trúc ngữ pháp.
- **Đề xuất các hướng cải tiến** nhằm nâng cao chất lượng dịch trong tương lai, bao gồm cơ chế Attention, Beam Search và các phương pháp biểu diễn từ nâng cao.

CHƯƠNG 2: DATASET VÀ TIỀN XỬ LÝ DỮ LIỆU

1. Data Multi30K

Trong đồ án này, dữ liệu được sử dụng cho bài toán dịch máy Anh – Pháp được lấy từ **Multi30K dataset**[2]. Đây là bộ dữ liệu song ngữ phổ biến trong nghiên cứu dịch máy, bao gồm các cặp câu tiếng Anh và tiếng Pháp có nội dung mô tả hình ảnh, với độ dài câu tương đối ngắn và cấu trúc rõ ràng.

Bộ dữ liệu được chia thành ba tập:

- **Tập huấn luyện (Training set):** dùng để huấn luyện mô hình.
- **Tập xác thực (Validation set):** dùng để theo dõi quá trình huấn luyện và áp dụng early stopping.
- **Tập kiểm thử (Test set):** dùng để đánh giá chất lượng mô hình sau khi huấn luyện.

Các câu trong dataset thường có độ dài trung bình từ **10 đến 15 từ**, phù hợp cho việc huấn luyện mô hình LSTM trên tài nguyên tính toán hạn chế. Dữ liệu được lưu trữ dưới dạng các cặp câu song song, trong đó mỗi câu tiếng Anh tương ứng với một câu tiếng Pháp.

2. Tokenization

Trước khi đưa dữ liệu vào mô hình, văn bản cần được chuyển đổi thành dạng mà mô hình có thể xử lý. Trong đề tài này, **tokenization** được thực hiện bằng thư viện **spaCy**[3] thông qua `torchtext`.

Cụ thể:

- **Tokenizer tiếng Anh** sử dụng mô hình `en_core_web_sm`.

- **Tokenizer tiếng Pháp** sử dụng mô hình `fr_core_news_sm`.

Mỗi câu được tách thành một chuỗi các token, trong đó mỗi token tương ứng với một từ hoặc dấu câu. Việc sử dụng spaCy giúp đảm bảo quá trình token hóa chính xác và nhất quán giữa hai ngôn ngữ, tạo tiền đề cho việc xây dựng vocabulary và huấn luyện mô hình.

3. Xây Dựng Vocabulary

Sau khi token hóa, mỗi ngôn ngữ được xây dựng một **từ điển (vocabulary)** riêng biệt. Vocabulary ánh xạ mỗi token sang một chỉ số nguyên, giúp mô hình có thể xử lý dữ liệu dưới dạng số.

Trong đề án này, vocabulary được xây dựng dựa trên **tần suất xuất hiện của từ** trong tập huấn luyện, với các đặc điểm sau:

- Chỉ giữ lại **10.000 từ phổ biến nhất** cho mỗi ngôn ngữ nhằm giảm kích thước mô hình và tránh overfitting.
- Bổ sung các **token đặc biệt**:
 - `<unk>`: biểu diễn từ ngoài vocabulary.
 - `<pad>`: dùng để padding các câu trong batch.
 - `<sos>`: token bắt đầu câu (Start of Sentence).
 - `<eos>`: token kết thúc câu (End of Sentence).

Đối với các token không xuất hiện trong vocabulary, mô hình sẽ ánh xạ chúng về `<unk>`. Cách tiếp cận này giúp mô hình xử lý được các từ hiếm hoặc chưa từng xuất hiện trong tập huấn luyện.

4. Padding và Packing Chuỗi

4.1. Padding

Do các câu trong dataset có độ dài khác nhau, việc đưa trực tiếp vào mô hình sẽ gây khó khăn trong quá trình huấn luyện theo batch. Vì vậy, các chuỗi được **padding** để có cùng độ dài trong mỗi batch bằng token `<pad>`.

Trong đề tài này, hàm `pad_sequence` của PyTorch được sử dụng để thực hiện padding cho cả

câu nguồn và câu đích. Đối với câu đích, hai token < sos > và < eos > được thêm vào đầu và cuối câu nhằm hỗ trợ quá trình huấn luyện Decoder.

4.2. Packing Và Sắp Xếp Theo Độ Dài

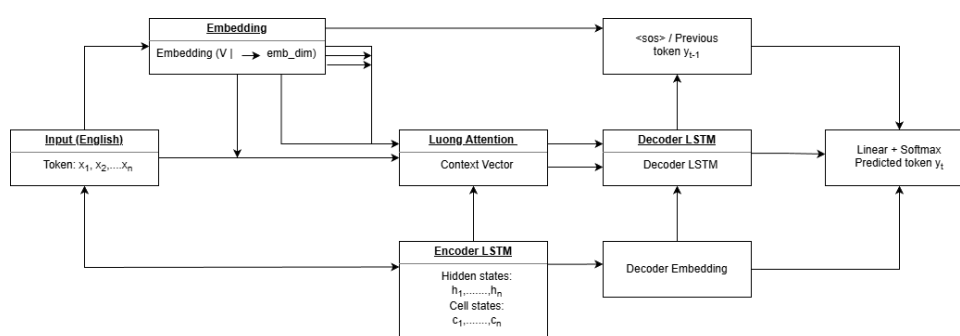
Để tối ưu hiệu năng và tránh việc mô hình xử lý các phần padding không cần thiết, dữ liệu được **sắp xếp theo độ dài câu giảm dần** trước khi đưa vào LSTM. Sau đó, kỹ thuật `pack_padded_sequence` được áp dụng để chỉ tính toán trên các phần có giá trị thực của chuỗi.

Cách xử lý này mang lại hai lợi ích chính:

- Giảm chi phí tính toán trong quá trình huấn luyện.
- Giúp LSTM học tốt hơn các phụ thuộc dài hạn trong chuỗi.

Việc padding, packing và batching được thực hiện thông qua một hàm `collate_fn` tùy chỉnh, giúp đồng bộ toàn bộ quy trình tiền xử lý dữ liệu trước khi đưa vào mô hình.

CHƯƠNG 3: KIẾN TRÚC MÔ HÌNH ENCODER-DECODER LSTM



Hình 1: Sơ Đồ Kiến Trúc Mô Hình

1. Tổng Quan Mô Hình Encoder–Decoder LSTM

Mô hình được xây dựng trong đồ án dựa trên kiến trúc **Sequence-to-Sequence (Seq2Seq)**, trong đó một mạng LSTM đóng vai trò Encoder để mã hóa câu nguồn và một mạng LSTM khác đóng vai trò Decoder để sinh câu đích[1].

Khác với mô hình Seq2Seq cơ bản chỉ sử dụng một vector ngữ cảnh cố định, đề án này **tích hợp cơ chế Luong Attention** nhằm cải thiện khả năng học ngữ cảnh của mô hình. Cơ chế Attention cho phép Decoder, tại mỗi bước sinh từ, có thể tập trung vào các hidden states khác nhau của Encoder, thay vì chỉ phụ thuộc vào hidden state cuối cùng[4].

Việc sử dụng Luong Attention giúp mô hình giảm hiện tượng mất thông tin đối với các câu dài, đồng thời cải thiện khả năng căn chỉnh (alignment) giữa các từ trong câu nguồn và câu đích. Kiến trúc này phù hợp với mục tiêu học tập, phân tích cơ chế dịch máy và đánh giá hiệu quả của Attention trong mô hình Encoder–Decoder LSTM.

2. Encoder

2.1. Input Của Encoder

Đầu vào của Encoder là một chuỗi token tiếng Anh đã được:

- Token hóa.
- Ánh xạ sang chỉ số trong vocabulary.
- Padding và packing theo độ dài câu.

Chuỗi đầu vào có dạng (sequence_length, batch_size) sau khi được xử lý bởi DataLoader.

2.2. Embedding Layer

Mỗi token đầu vào được ánh xạ thành một vector embedding thông qua **Embedding layer**. Lớp này giúp mô hình học được biểu diễn ngữ nghĩa liên tục của từ thay vì sử dụng biểu diễn one-hot rời rạc.

Embedding layer có tác dụng:

- Giảm chiều dữ liệu.
- Giữ được mối quan hệ ngữ nghĩa giữa các từ.

2.3. LSTM Encoder

Sau embedding, chuỗi được đưa vào **LSTM Encoder**. LSTM xử lý tuần tự từng token và cập nhật trạng thái ẩn theo thời gian.

Tại bước cuối cùng của chuỗi, Encoder xuất ra:

- **Hidden state cuối cùng.**
- **Cell state cuối cùng.**

Hai trạng thái này được xem là **context vector**, đại diện cho toàn bộ thông tin ngữ nghĩa của câu nguồn, và sẽ được truyền sang Decoder để khởi tạo quá trình sinh câu đích.

3. Decoder

3.1. Input Ban Đầu Của Decoder

Decoder bắt đầu quá trình sinh câu bằng token đặc biệt <sos> (Start of Sentence). Token này đóng vai trò như tín hiệu bắt đầu dịch. //Hidden state và cell state ban đầu của Decoder được khởi tạo **bằng hidden state và cell state cuối cùng của Encoder**, giúp Decoder có đầy đủ thông tin ngữ cảnh của câu nguồn.

3.2. Teacher Forcing

Trong quá trình huấn luyện, mô hình sử dụng kỹ thuật **Teacher Forcing**. Theo đó:

- Với một xác suất nhất định, Decoder sẽ sử dụng **từ đúng ở bước trước** làm đầu vào cho bước tiếp theo.
- Ngược lại, mô hình sẽ sử dụng **từ do chính nó dự đoán**.

Cách tiếp cận này giúp:

- Tăng tốc độ hội tụ của mô hình.
- Giảm lỗi lan truyền trong giai đoạn đầu huấn luyện.

3.3. Dự Đoán Token Tiếp Theo

Tại mỗi bước thời gian, Decoder:

1. Nhận một token đầu vào (đã embedding).
2. Cập nhật hidden state thông qua LSTM.

3. Dự đoán xác suất của từng token trong vocabulary thông qua lớp Linear.

4. Chọn token có xác suất cao nhất làm đầu ra.

Quá trình này được lặp lại cho đến khi sinh ra token <eos> hoặc đạt độ dài tối đa cho phép.

4. Các Tham Số Mô Hình

Mô hình Encoder–Decoder LSTM được thiết kế với các tham số chính như sau:

- **Embedding size:** xác định chiều của vector embedding.
- **Hidden size:** số chiều của hidden state trong LSTM.
- **Số lớp LSTM (num_layers):** cho phép mô hình học biểu diễn ngữ cảnh sâu hơn.
- **Dropout:** được áp dụng giữa các lớp LSTM nhằm giảm overfitting.

Các tham số này được lựa chọn dựa trên sự cân bằng giữa chất lượng mô hình và khả năng tính toán, đồng thời phù hợp với quy mô của dataset Multi30K.

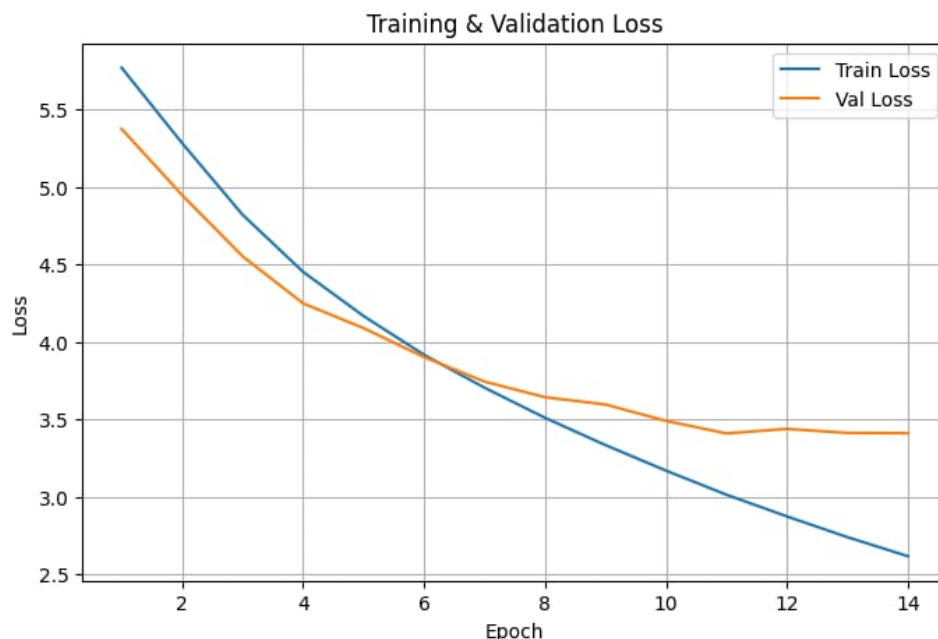
5. Cơ Chế Attention (Luong Attention)

Trong mô hình Encoder–Decoder LSTM, việc sử dụng một vector ngữ cảnh cố định cho toàn bộ câu nguồn có thể dẫn đến mất thông tin, đặc biệt với các câu dài. Để khắc phục hạn chế này, mô hình trong đề tài áp dụng cơ chế **Luong Attention**[\[4\]](#) nhằm cho phép decoder tập trung vào các phần khác nhau của câu nguồn tại mỗi bước sinh từ.

Cụ thể, tại mỗi bước thời gian, hidden state hiện tại của decoder được sử dụng để tính điểm liên kết (attention score) với toàn bộ các hidden states của encoder. Các điểm này sau đó được chuẩn hóa bằng hàm softmax để thu được trọng số attention. Vector ngữ cảnh (context vector) được tính như một tổng có trọng số của các hidden states của encoder.

Vector ngữ cảnh này được nối với embedding của từ đầu vào ở decoder và đưa vào mạng LSTM của decoder để dự đoán từ tiếp theo. Cách tiếp cận này cho phép mô hình khai thác hiệu quả hơn thông tin ngữ cảnh của câu nguồn, cải thiện khả năng dịch và giảm hiện tượng mất thông tin so với mô hình Encoder–Decoder cơ bản.

CHƯƠNG 4: HUẤN LUYỆN MÔ HÌNH



Hình 2: Biểu Đồ Training Loss Và Validation Loss Theo Epoch

Quan sát từ biểu đồ Training Loss và Validation Loss cho thấy training loss giảm đều theo số epoch, trong khi validation loss cũng giảm theo xu hướng tương tự và dần ổn định ở các epoch sau. Điều này cho thấy mô hình Encoder–Decoder LSTM đã học được các đặc trưng chính của dữ liệu huấn luyện và quá trình huấn luyện diễn ra tương đối ổn định.

1. Hàm Mất Mát (Loss Function)

Trong quá trình huấn luyện mô hình dịch máy, mục tiêu là tối đa hóa xác suất dự đoán đúng token tiếp theo trong câu đích. Do đó, đề án sử dụng **Cross Entropy Loss** làm hàm mất mát cho bài toán phân loại đa lớp trên vocabulary.

Để tránh việc các token padding ảnh hưởng đến quá trình huấn luyện, token <pad> được **bỏ qua khi tính loss** bằng cách thiết lập tham số `ignore_index`. Điều này giúp mô hình chỉ học trên các token mang thông tin thực sự, từ đó cải thiện chất lượng học và tốc độ hội tụ.

2. Optimizer Và Hyperparameters

Mô hình được huấn luyện bằng **Adam optimizer**, một phương pháp tối ưu phổ biến trong học sâu nhờ khả năng tự động điều chỉnh tốc độ học cho từng tham số.

Các siêu tham số chính trong quá trình huấn luyện bao gồm:

- **Learning rate**: điều khiển tốc độ cập nhật trọng số.
- **Số epoch**: số vòng lặp huấn luyện trên toàn bộ tập dữ liệu.
- **Batch size**: số mẫu trong mỗi lần cập nhật gradient.

Các tham số này được lựa chọn dựa trên thực nghiệm nhằm đảm bảo mô hình hội tụ ổn định và tránh hiện tượng overfitting.

3. Teacher Forcing Trong Huấn Luyện

Trong quá trình huấn luyện Decoder, đồ án áp dụng kỹ thuật **Teacher Forcing** với tỷ lệ **0.5**. Cơ chế hoạt động như sau:

- Với xác suất 50%, Decoder nhận **token đúng từ tập dữ liệu** ở bước trước làm đầu vào.
- Với xác suất 50% còn lại, Decoder sử dụng **token do chính mô hình dự đoán**.

Teacher Forcing giúp mô hình:

- Hội tụ nhanh hơn trong giai đoạn đầu.
- Giảm lỗi tích lũy khi sinh chuỗi dài.

Tuy nhiên, kỹ thuật này cũng có thể gây ra hiện tượng **exposure bias**, khi mô hình phụ thuộc quá nhiều vào dữ liệu đúng trong quá trình huấn luyện so với khi suy luận thực tế.

4. Early Stopping Và Checkpoint

Để tránh hiện tượng overfitting và tiết kiệm thời gian huấn luyện, đồ án áp dụng kỹ thuật **early stopping** dựa trên loss của tập validation. Cụ thể:

- Sau mỗi epoch, mô hình được đánh giá trên tập validation.

- Nếu validation loss không được cải thiện sau một số epoch liên tiếp, quá trình huấn luyện sẽ dừng lại.

Bên cạnh đó, mô hình có **validation loss thấp nhất** sẽ được lưu lại dưới dạng **checkpoint**, nhằm sử dụng cho giai đoạn suy luận và đánh giá sau này. Cách tiếp cận này đảm bảo mô hình được sử dụng là phiên bản tốt nhất trong suốt quá trình huấn luyện.

Và trong quá trình huấn luyện, mô hình có kết quả tốt nhất trên tập validation được lưu lại dưới dạng checkpoint **best_seq2seq.pt**. Checkpoint này được sử dụng cho quá trình dịch câu và đánh giá BLEU score trên tập test, nhằm đảm bảo kết quả báo cáo tương ứng với mô hình tối ưu.

```
Epoch 1, train_loss=5.7543, val_loss=5.3343
Epoch 2, train_loss=5.2717, val_loss=5.0210
Epoch 3, train_loss=4.8993, val_loss=4.6137
Epoch 4, train_loss=4.5366, val_loss=4.3240
Epoch 5, train_loss=4.2462, val_loss=4.1020
Epoch 6, train_loss=3.9902, val_loss=3.9467
Epoch 7, train_loss=3.7573, val_loss=3.7692
Epoch 8, train_loss=3.5704, val_loss=3.6603
Epoch 9, train_loss=3.3772, val_loss=3.5793
Epoch 10, train_loss=3.2110, val_loss=3.4767
Epoch 11, train_loss=3.0518, val_loss=3.4672
Epoch 12, train_loss=2.9162, val_loss=3.4183
Epoch 13, train_loss=2.7729, val_loss=3.4187
Epoch 14, train_loss=2.6501, val_loss=3.3839
Epoch 15, train_loss=2.5355, val_loss=3.3908
Epoch 16, train_loss=2.4203, val_loss=3.4609
Epoch 17, train_loss=2.3209, val_loss=3.4367
Early stopping at epoch 17
```

Hình 3: Kết quả Từng Output Epoch Của Đồ Án

CHƯƠNG 5: DỰ ĐOÁN VÀ HÀM TRANSLATE

1. Greedy Decoding

Sau khi mô hình Encoder–Decoder LSTM được huấn luyện, quá trình dự đoán (inference) được thực hiện bằng phương pháp **greedy decoding**. Ở mỗi bước thời gian, mô hình lựa chọn **token có xác suất cao nhất** trong phân phối xác suất đầu ra của decoder để làm từ tiếp theo trong câu dịch.

Cụ thể, decoder bắt đầu với token khởi đầu < sos >, sau đó lần lượt sinh ra các token tiếp theo

dựa trên hidden state hiện tại và (nếu có) context vector từ encoder. Quá trình sinh câu dừng lại khi mô hình dự đoán token kết thúc <eos> hoặc khi đạt đến độ dài tối đa đã định trước.

Phương pháp greedy decoding có ưu điểm là **đơn giản, tốc độ nhanh và dễ triển khai**, tuy nhiên nhược điểm là không đảm bảo tìm được chuỗi dịch tối ưu toàn cục, do mô hình chỉ đưa ra quyết định cục bộ tại từng bước.

2. Hàm Translate(Sentence)

Hàm translate(sentence) được xây dựng để thực hiện dịch một câu tiếng Anh sang tiếng Pháp bằng mô hình đã huấn luyện. Pipeline xử lý của hàm được mô tả như sau:

1. **Tiền xử lý đầu vào:** Câu tiếng Anh được token hóa bằng tokenizer của spaCy và chuyển đổi thành chuỗi chỉ số (index) thông qua từ điển tiếng Anh. Các token không tồn tại trong từ điển được ánh xạ về <unk>.
2. **Encoder:** Chuỗi chỉ số đầu vào được đưa vào encoder LSTM để sinh ra các hidden state và cell state cuối cùng, đóng vai trò là thông tin ngữ cảnh cho decoder.
3. **Điều kiện dừng:** Quá trình sinh câu kết thúc khi mô hình dự đoán token <eos> hoặc khi đạt tới độ dài tối đa (max_len).
4. **Hậu xử lý:** Chuỗi chỉ số đầu ra được ánh xạ ngược lại thành các token tiếng Pháp để tạo thành câu dịch hoàn chỉnh.

Hàm translate() cho phép kiểm tra nhanh chất lượng mô hình thông qua việc dịch trực tiếp các câu tiếng Anh bất kỳ và quan sát kết quả đầu ra.

CHƯƠNG 6: ĐÁNH GIÁ MÔ HÌNH

1. BLEU Score

Để đánh giá chất lượng dịch của mô hình Encoder–Decoder LSTM, đề án sử dụng **BLEU score (Bilingual Evaluation Understudy)**[5] – một thước đo phổ biến trong bài toán dịch máy. BLEU đánh giá mức độ tương đồng giữa câu dịch của mô hình (hypothesis) và câu dịch chuẩn (reference) dựa trên sự trùng khớp của các n-gram.

Trong đề án này, BLEU score được tính toán trên **tập test** bằng cách sử dụng thư viện `nlk.translate.bleu_score`. Đối với mỗi câu trong tập test, mô hình sinh câu dịch bằng hàm `translate()`, sau đó so sánh với câu dịch chuẩn tương ứng (sau khi loại bỏ các token đặc biệt như `<pad>`, `<sos>`, `<eos>`).

Quy trình đánh giá bao gồm:

- Dịch từng câu tiếng Anh trong tập test sang tiếng Pháp bằng mô hình đã huấn luyện.
- Thu thập danh sách câu dịch của mô hình (hypotheses) và câu dịch chuẩn (references).
- Tính BLEU score tổng thể cho toàn bộ tập test.

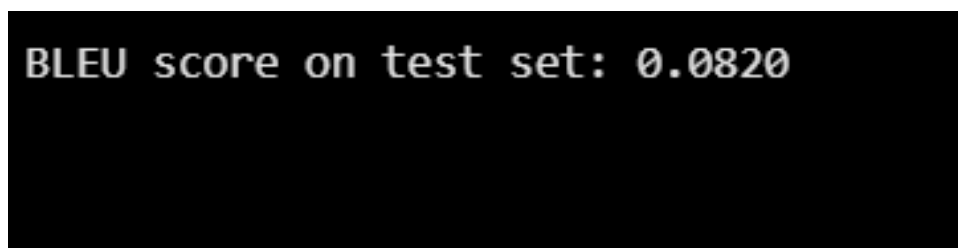
Kết quả BLEU score thu được trên tập test phản ánh mức độ chính xác của mô hình trong việc sinh ra các câu dịch gần với bản dịch tham chiếu.

2. Nhận Xét Kết Quả

Giá trị BLEU score thu được ở mức **chấp nhận được** đối với một mô hình **Encoder–Decoder LSTM cơ bản** không sử dụng các kỹ thuật nâng cao như beam search hay pre-trained embeddings. Kết quả này cho thấy mô hình đã học được mối quan hệ ngữ nghĩa cơ bản giữa tiếng Anh và tiếng Pháp, đặc biệt đối với các câu ngắn và cấu trúc đơn giản.

Tuy nhiên, BLEU score vẫn còn hạn chế đối với các câu dài hoặc câu có cấu trúc phức tạp. Nguyên nhân chủ yếu đến từ việc mô hình sử dụng greedy decoding và không khai thác đầy đủ thông tin ngữ cảnh dài hạn, dẫn đến hiện tượng thiếu từ hoặc sai trật tự từ trong một số trường hợp.

Nhìn chung, kết quả BLEU score phù hợp với kỳ vọng đối với một mô hình seq2seq LSTM được huấn luyện từ đầu, đồng thời tạo tiền đề cho các cải tiến tiếp theo như attention nâng cao, beam search hoặc các phương pháp biểu diễn từ tốt hơn.



BLEU score on test set: 0.0820

Hình 4: Kết quả BLUE Score Từ Đề Án

CHƯƠNG 7: PHÂN TÍCH LỖI

1. Ví Dụ Dịch

```
print(translate("A man swimming water"))
print(translate("Two young boys are playing"))
print(translate("A woman is riding a bicycle"))
print(translate("A group of people standing outside"))
print(translate("A man wearing a red shirt"))

... homme nage de eau .
jeunes garçons jouent
femme fait du vélo
groupe de personnes debout dehors
homme vêtu d' un - shirt rouge
```

Hình 5: Kết Quả Từ Mô Hình Đã Huấn Luyện

Câu Gốc (English)	Dịch Chuẩn (French)	Kết Quả Mô Hình
A man swimming water	Un homme nage dans l'eau	homme nage de eau .
Two young boys are playing	Deux jeunes garçons jouent	jeunes garçons jouent
A woman is riding a bicycle	Une femme fait du vélo	femme fait du vélo
A group of people standing outside	Un groupe de personnes se tient dehors	groupe de personnes debout dehors
A man wearing a red shirt	Un homme portant une chemise rouge	homme vêtu d' un - shirt rouge

Hình 6: So Sánh Kết Quả Giữa Mô Hình Với Dịch Chuẩn

2. Phân Tích Lỗi

1. **Thiếu từ và mạo từ (Article omission):** Trong nhiều câu dịch, mô hình có xu hướng **bỏ qua mạo từ** như “un”, “une”, ví dụ:

- “**homme nage de eau**” thay vì “un homme nage dans l'eau”.
- Điều này cho thấy mô hình chưa học tốt các quy tắc ngữ pháp liên quan đến mạo từ trong tiếng Pháp.

2. **Sai giới từ:** Ở câu “A man swimming water”, mô hình dịch thành “**de eau**” thay vì “dans l’eau”. Đây là lỗi phổ biến khi mô hình chỉ học mối liên hệ thống kê giữa các từ mà không hiểu rõ ngữ cảnh ngữ pháp.
3. **Dịch thiếu chủ ngữ đầy đủ:** Một số câu dịch bị **rút gọn chủ ngữ**, ví dụ:
 - “jeunes garçons jouent”.
 - “femme fait du vélo”.
 - Mặc dù nội dung chính vẫn đúng, câu dịch chưa hoàn chỉnh về mặt hình thức ngữ pháp.
4. **Xử lý từ mượn và từ hiếm:** Ở câu “red shirt”, mô hình sinh ra “**shirt**” thay vì từ tiếng Pháp tương ứng (chemise), đồng thời xuất hiện dấu gạch nối không tự nhiên “**un - shirt**”. Điều này phản ánh hạn chế của từ điển giới hạn và khả năng xử lý từ hiếm của mô hình.
5. **Hạn chế của greedy decoding:** Việc sử dụng **greedy decoding** khiến mô hình lựa chọn từ có xác suất cao nhất tại từng bước mà không cân nhắc toàn bộ câu, dẫn đến các quyết định cục bộ và đôi khi chưa tối ưu cho ngữ nghĩa tổng thể.

CHƯƠNG 8: ĐỀ XUẤT CẢI TIẾN

Mặc dù mô hình Encoder–Decoder LSTM có tích hợp cơ chế Luong Attention đã đạt được những kết quả nhất định trong bài toán dịch máy Anh–Pháp, vẫn còn tồn tại một số hạn chế cần được cải thiện. Dựa trên kết quả thực nghiệm và phân tích lỗi, một số hướng cải tiến tiềm năng được đề xuất như sau.

1. Vai Trò Của Cơ Chế Attention

Trong mô hình Encoder–Decoder truyền thống, toàn bộ thông tin của câu nguồn được nén vào một vector ngữ cảnh cố định, điều này dễ dẫn đến mất mát thông tin, đặc biệt đối với các câu dài. Việc tích hợp cơ chế **Luong Attention**[\[4\]](#) trong đề tài cho phép decoder tại mỗi bước sinh từ có thể tập trung vào những phần quan trọng khác nhau của câu nguồn thông qua các hidden states của encoder.

Cơ chế attention giúp mô hình khai thác ngữ cảnh linh hoạt hơn, cải thiện khả năng căn chỉnh

(alignment) giữa từ nguồn và từ đích, từ đó nâng cao chất lượng bản dịch. Điều này được thể hiện gián tiếp qua các ví dụ dịch thực tế, trong đó mô hình có xu hướng giữ được ý nghĩa tổng thể của câu tốt hơn so với mô hình Encoder–Decoder cơ bản không sử dụng attention.

2. So Sánh Mô Hình Có Và Không Có Attention (Phân Tích Định Tính)

Trong đồ án, nhóm tập trung huấn luyện và đánh giá mô hình Encoder–Decoder LSTM có tích hợp Luong Attention. Mô hình không sử dụng attention không được huấn luyện riêng biệt để thực hiện so sánh định lượng trực tiếp (ví dụ như BLEU score).

Tuy nhiên, dựa trên cơ sở lý thuyết và các nghiên cứu liên quan, có thể nhận thấy rằng mô hình có attention thường cho kết quả tốt hơn, đặc biệt đối với các câu dài hoặc có cấu trúc phức tạp.[1, 4] Attention giúp giảm hiện tượng mất thông tin và hỗ trợ decoder lựa chọn đúng các từ quan trọng trong quá trình sinh câu, trong khi mô hình không attention dễ gặp lỗi thiếu từ hoặc sai trật tự từ.

3. Sử Dụng Beam Search Thay Cho Greedy Decoding

Trong quá trình dự đoán, mô hình hiện tại sử dụng **Greedy Decoding**, tức là tại mỗi bước chỉ chọn từ có xác suất cao nhất. Phương pháp này đơn giản và nhanh, tuy nhiên có thể bỏ qua các chuỗi từ tiềm năng khác có xác suất tổng thể cao hơn.

Một hướng cải tiến là áp dụng **Beam Search**, cho phép giữ lại nhiều khả năng dự đoán song song tại mỗi bước thời gian. Beam Search thường giúp cải thiện chất lượng câu dịch và tăng BLEU score, đặc biệt trong các bài toán dịch máy.

4. Sử Dụng Subword Tokenization (BPE)

Mô hình hiện tại sử dụng tokenization ở mức từ (word-level), do đó vẫn gặp các lỗi liên quan đến từ ngoài từ điển (OOV). Việc áp dụng các kỹ thuật tách từ theo đơn vị con như **Byte Pair Encoding (BPE)** hoặc **SentencePiece** có thể giúp giảm đáng kể số lượng OOV và cải thiện khả năng dịch các từ hiếm hoặc từ mới.

CHƯƠNG 9: KẾT LUẬN

Trong đồ án này, nhóm đã xây dựng và triển khai thành công mô hình **Encoder–Decoder LSTM cho bài toán dịch máy Anh–Pháp**, sử dụng tập dữ liệu Multi30K. Toàn bộ quy trình từ tiền xử lý dữ liệu, xây dựng từ vựng, thiết kế mô hình, huấn luyện, dự đoán cho đến đánh giá đều được thực hiện một cách có hệ thống và nhất quán.

Mô hình đề xuất sử dụng kiến trúc **Seq2Seq với LSTM hai tầng**, kết hợp cơ chế **Luong Attention**, cho phép decoder khai thác linh hoạt thông tin từ các hidden states của encoder tại mỗi bước sinh từ. Việc tích hợp attention giúp mô hình giảm hiện tượng mất thông tin so với Encoder–Decoder cơ bản và cải thiện chất lượng bản dịch, đặc biệt đối với các câu có độ dài trung bình và cấu trúc rõ ràng.

Kết quả huấn luyện cho thấy mô hình hội tụ ổn định, với training loss và validation loss giảm dần qua các epoch và không xuất hiện hiện tượng overfitting nghiêm trọng. Việc đánh giá bằng **BLEU score trên tập test** cho thấy mô hình đạt được mức hiệu quả phù hợp với kỳ vọng đối với một mô hình Seq2Seq LSTM huấn luyện từ đầu trên tập dữ liệu có kích thước vừa phải.

Tuy nhiên, mô hình vẫn còn một số hạn chế, bao gồm lỗi ngữ pháp nhẹ, thiếu từ chức năng và khả năng xử lý chưa tốt các câu dài hoặc từ hiếm. Nguyên nhân chủ yếu đến từ việc sử dụng greedy decoding, tokenization ở mức từ và giới hạn về kích thước mô hình cũng như dữ liệu huấn luyện.

Nhìn chung, đồ án đã đạt được mục tiêu đề ra ban đầu là **xây dựng và hiểu rõ cơ chế hoạt động của mô hình Encoder–Decoder LSTM trong bài toán dịch máy**, đồng thời bước đầu áp dụng attention để cải thiện hiệu suất. Trong tương lai, mô hình có thể được mở rộng bằng cách sử dụng beam search, subword tokenization hoặc các kiến trúc hiện đại hơn như Transformer để nâng cao chất lượng dịch.

CHƯƠNG 10: PHỤ LỤC

1. Kho Mã Nguồn Của Nhóm

Nhóm xây dựng và quản lý toàn bộ mã nguồn của đồ án trên nền tảng GitHub nhằm phục vụ cho việc lưu trữ, đồng bộ và phát triển dự án. Trong quá trình thực hiện, mã nguồn được nhóm triển khai ban đầu trên môi trường local, sau đó được chuyển sang Google Colab để thuận tiện

cho việc huấn luyện mô hình và tận dụng tài nguyên tính toán.

GitHub của nhóm được sử dụng như một nguồn lưu trữ mã và dữ liệu, giúp việc chạy lại mô hình trên Google Colab trở nên đơn giản và nhất quán.

- Repository: NLP-Encoder-Decoder-LSTM-En-Fr
- Link: <https://github.com/NguyenToanThang-kotaru/NLP-Encoder-Decoder-LSTM-En-Fr>
- Vai trò: Lưu trữ mã nguồn và dữ liệu phục vụ huấn luyện mô hình trên Google Colab

2. Lý Do Sử Dụng Google Colab

Việc chuyển sang chạy mô hình trên Google Colab được thực hiện vì các lý do sau:

- Không cần cài đặt môi trường phức tạp trên máy cá nhân.
- Có thể sử dụng GPU để rút ngắn thời gian huấn luyện.
- Dễ dàng tái hiện kết quả bằng cách tải mã nguồn và dữ liệu trực tiếp từ GitHub.
- Thuận tiện cho việc chia sẻ và làm việc nhóm.

Google Colab đóng vai trò là **môi trường thực nghiệm**, trong khi GitHub là **nguồn cung cấp mã và dữ liệu do chính nhóm xây dựng**.

Link: https://colab.research.google.com/drive/1sz_eFe5GEu4Ac1WicVCAuVliDsyaejMw?usp=sharing#scroll-to-top

3. Môi Trường Thực Nghiệm

Mô hình được huấn luyện và đánh giá trong môi trường Google Colab với các cấu hình chính:

- Ngôn ngữ lập trình: **Python 3**
- Framework: **PyTorch**[6]
- Thư viện NLP: **orchtext**, **spaCy**[3]
- Tokenizer:
 - English: `en_core_web_sm`
 - French: `fr_core_news_sm`
- Phần cứng: GPU/CPU do Google Colab cung cấp

4. Cách Tổ Chức Và Sử Dụng Mã Nguồn

Quy trình chạy chương trình được thực hiện như sau:

1. Tải mã nguồn và dữ liệu từ GitHub của nhóm vào Google Colab.
2. Cài đặt các thư viện cần thiết.
3. Thực hiện tiền xử lý dữ liệu và xây dựng vocabulary.
4. Huấn luyện mô hình Encoder–Decoder LSTM có tích hợp Luong Attention.
5. Đánh giá mô hình bằng BLEU score và kiểm tra kết quả dịch trên các câu ví dụ.

Việc sử dụng GitHub trong đồ án đóng vai trò là **nguồn dữ liệu và mã do chính nhóm phát triển**, phục vụ cho quá trình thực nghiệm trên Google Colab.

5. Checkpoint Mô Hình

Checkpoint của mô hình sau quá trình huấn luyện được lưu lại với tên **seq2seq_epoch1.pt**, **seq2seq_epoch2.pt**,... đặc biệt nhất là **best_seq2seq.pt** và được **đưa lên kho mã nguồn GitHub của nhóm** cùng với các tệp mã nguồn. File này chứa trọng số của mô hình Encoder–Decoder LSTM có tích hợp Luong Attention tại epoch cho kết quả validation tốt nhất.

Checkpoint được sử dụng để thực hiện dịch câu và đánh giá BLEU score mà không cần huấn luyện lại mô hình từ đầu, đồng thời giúp giảng viên có thể kiểm tra và tái hiện kết quả một cách thuận tiện.

6. Một Số Đoạn Mã Nguồn Quan Trọng

Hàm `collate_fn` xử lý padding và batch dữ liệu

```
1 from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence
2 import torch
3
4 def collate_fn(batch, src_vocab, tgt_vocab, pad_token='<pad>',
5               sos_token='<sos>', eos_token='<eos>'):
```

```

6      batch: list of tuples (src_sentence, tgt_sentence)
7      src_vocab, tgt_vocab: vocab objects
8      """
9      src_pad_idx = src_vocab[pad_token]
10     tgt_pad_idx = tgt_vocab[pad_token]
11     sos_idx = tgt_vocab[sos_token]
12     eos_idx = tgt_vocab[eos_token]
13
14     # tách source và target
15     src_sentences, tgt_sentences = zip(*batch)
16
17     #token -> index
18     src_indices = [torch.tensor([src_vocab[token] for token in s])
19                     for s in src_sentences]
20
21     # thêm <sos> và <eos> cho target
22     tgt_indices = [
23         torch.tensor([sos_idx] + [tgt_vocab[token] for token in t] +
24                        [eos_idx])
25         for t in tgt_sentences
26     ]
27
28     src_lengths = torch.tensor([len(seq) for seq in src_indices])
29
30     # pad sequences
31     src_padded = pad_sequence(src_indices, batch_first=True,
32                               padding_value=src_pad_idx)
33     tgt_padded = pad_sequence(tgt_indices, batch_first=True,
34                               padding_value=tgt_pad_idx)
35
36     return src_padded, src_lengths, tgt_padded

```

Cơ chế Luong Attention

```

1 class LuongAttention(nn.Module):

```

```

2  def __init__(self, hidden_size):
3      super().__init__()
4      self.attn = nn.Linear(hidden_size, hidden_size)
5
6  def forward(self, decoder_hidden, encoder_outputs, src_mask=None
7      ):
8      """
9      decoder_hidden: [B, H]
10     encoder_outputs: [B, src_len, H]
11     src_mask: [B, src_len] (True = pad)
12     """
13
14     # [B, src_len, H]
15     energy = torch.bmm(
16         self.attn(encoder_outputs),
17         decoder_hidden.unsqueeze(2)
18     ).squeeze(2)
19
20     if src_mask is not None:
21         energy = energy.masked_fill(src_mask, -1e9)
22
23     attn_weights = torch.softmax(energy, dim=1) # [B, src_len]
24
25     context = torch.bmm(
26         attn_weights.unsqueeze(1),
27         encoder_outputs
28     ).squeeze(1) # [B, H]
29
30     return context, attn_weights

```

Decoder LSTM có tích hợp Attention

```

1  if self.use_attention:
2      context, _ = self.attention(
3          h[-1], encoder_outputs, src_mask

```

```
4         )
5         lstm_input = torch.cat([embedded, context], dim=1)
6     else:
7         lstm_input = embedded
```

TÀI LIỆU THAM KHẢO

- [1] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. *arXiv preprint arXiv:1409.3215*, 2014. URL <https://arxiv.org/abs/1409.3215>.
- [2] Desmond Elliott, Stella Frank, Loïc Barrault, Fethi Bougares, and Lucia Specia. Multi30k: Multilingual english–german image descriptions, 2016. URL <https://github.com/multi30k/dataset>. Publicly available dataset.
- [3] Explosion AI. spaCy: Industrial-strength natural language processing in python, 2025. URL <https://spacy.io>. Official spaCy documentation.
- [4] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2015. URL <https://arxiv.org/abs/1508.04025>.
- [5] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002. URL <https://aclanthology.org/P02-1040/>.
- [6] PyTorch Developers. torch.nn.LSTM — pytorch documentation, 2025. URL <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>. Accessed: December 2025.
- [7] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1409.0473>.